

Database Normalization Justification

The Coward King

Jose Abel Dominguez Rish

Leonardo Andre Flores Mendoza

Nicolas Casillas Larranaga

1. Importance of Normalization

Database normalization is the process of organizing a relational database to reduce data redundancy and improve data integrity. By following normal forms, we ensure that each piece of information is stored in exactly one place, which makes the database easier to maintain, update, and scale. For The Coward King, normalization was especially important because the game involves multiple interrelated entities: players, runs, cards, enemies, hordes, and upgrades. Without normalization, changes to a card's stats would require updates in multiple places, and inconsistencies could cause bugs in gameplay.

2. Normalization Applied

We applied normalization principles up to the Third Normal Form (3NF) across all tables:

First Normal Form (1NF): All tables contain only atomic values. No field stores lists or repeating groups. For example, the Card table stores one value per cell (name, type, base_ap_cost, base_hp, base_damage, range_attack, duration, effect_description).

Second Normal Form (2NF): All non-key attributes depend fully on the primary key. Tables with composite keys, such as Run_Hand (run_id, horde_id, card_id), were verified so that every attribute (is_retained) depends on the full key, not a subset.

Third Normal Form (3NF): Transitive dependencies were identified and removed. For example, enemy stats (hp, damage, speed) are stored in the Enemy table rather than duplicated inside Run or Horde. Card stats are stored once in Card and referenced by foreign key everywhere else.

3. Justified Exception to 3NF

The Level table includes a boss_name column, which technically creates a transitive dependency since the boss name could be derived by querying the Enemy table (WHERE level_id = X AND is_boss = TRUE). The team decided to keep this column

for practical reasons: it simplifies queries during gameplay, avoids an extra JOIN on every level load, and improves readability of the schema for team members. Since boss names never change at runtime, there is no risk of data inconsistency from this denormalization.

4. Contributions of Generative AI to the ER Diagram

We used Claude as a complementary tool during the database design process. The AI contributed in the following ways:

Schema structure: The AI recommended separating the run progression data into two tables, Run_Hand (cards dealt per horde) and Run_Horde (turns survived, enemies killed per horde) instead of storing all run data in a single table. This avoided repeating groups and kept each table focused on one responsibility.

Foreign key ordering: The AI flagged that Run_Hand and Card_Upgrade must be dropped before Card when resetting the schema, due to foreign key constraints. This helped the team avoid errors during development.

It is important to note that the AI didn't replace the developers decision making. All final design choices were made by us, based on the project requirements and concepts learned in class.

5. Summary

The database for The Coward King was designed following 3NF principles, with one justified exception in the Level table. Normalization ensured data integrity across all player sessions, card upgrades, and run progression. Generative AI tools contributed specific technical improvements without replacing the conceptual work of the development team.